SỔ TAY THỰC HÀNH / 2026

AWESOME ATLAS LAB

Từ bản đồ tài nguyên đến danh sách tự tuyển chọn

TÌM → KIỂM CHỨNG → CHỌN → DUY TRÌ

27 nhóm nội dung
8 chương · 16 bài học có đầu ra

Tự tìm đúng nguồn, tự ghi bằng chứng, tự xây danh sách.
Đây là giáo trình độc lập về cách dùng awesome.

moda.app

Cách dùng sổ tay

Giáo trình biên soạn riêng: bản đồ đủ 27 nhóm, 8 chương, 16 bài thực hành về dùng và xây danh sách. Không phải giáo trình chính thức của awesome và không thay thế hướng dẫn kỹ thuật của mọi repo được liên kết.

01 / KẾT QUẢ MONG MUỐN

Bạn tìm được tài nguyên đúng nhu cầu, đánh giá được bằng chứng và tạo được danh sách cá nhân có lý do tuyển chọn.

02 / CHUẨN BỊ

Trình duyệt và nơi ghi chú là đủ cho phần cốt lõi. Tài khoản GitHub cần khi lưu repo hoặc đề xuất thay đổi; Node.js và Git chỉ cần cho bài lint.

03 / NHỊP HỌC

Đọc trang chương → làm hai bài → lưu đầu ra → tự kiểm tra. Có thể quay lại nguồn gốc bằng nhãn [R1]–[R6] ở cuối trang.

04 / PHÂN BIỆT NGUỒN VỚI BÀI TẬP

Mô tả repo và quy định cộng đồng có trích dẫn. Câu hỏi, rubric, ví dụ và lộ trình là cách biên soạn của người viết.

05 / GIỚI HẠN

Không phải hướng dẫn triển khai mọi công nghệ trong 27 nhóm. Hướng dẫn kỹ thuật chi tiết nằm ở tài liệu của từng tài nguyên đích.

16 bài thực hành

Bấm tên bài để mở bản đọc trên GitHub Radar. Bản PDF tải về có liên kết nhảy tới trang bài học.

CHƯƠNG 1 / Hiểu đúng Awesome

01. Đặt câu hỏi trước khi mở danh sách tr. 7
02. Đi qua đúng ba tầng liên kết tr. 8

CHƯƠNG 2 / Đọc bản đồ 27 nhóm

03. Chọn nhóm theo việc cần làm tr. 10
04. Biến danh sách thành lộ trình học tr. 11

CHƯƠNG 3 / Tuyển chọn bằng bằng chứng

05. Lập hồ sơ ứng viên tr. 13
06. So sánh và loại có lý do tr. 14

CHƯƠNG 4 / Viết mục giới thiệu có ích

07. Viết một entry không quảng cáo tr. 16
08. Thử trải nghiệm của người đọc tr. 17

CHƯƠNG 5 / Tự xây danh sách của mình

09. Tạo README có phạm vi và mục lục tr. 19
10. Thêm quy tắc đóng góp và giấy phép tr. 20

CHƯƠNG 6 / Kiểm tra chất lượng

11. Rà soát thủ công trước khi chạy công cụ tr. 22
12. Chạy awesome-lint và hiểu kết quả tr. 23

CHƯƠNG 7 / Đóng góp đúng nơi, đúng điều kiện

13. Đề xuất một tài nguyên vào danh sách có sẵn tr. 25
14. Kiểm tra điều kiện cho một danh sách mới tr. 26

CHƯƠNG 8 / Duy trì và tự đánh giá

15. Thiết kế vòng bảo trì tr. 28
16. Đồ án: danh sách nhỏ có bằng chứng tr. 29

moda.app

Bản đồ 27 nhóm / 01

Chọn nhóm theo nhiệm vụ. Ví dụ lấy từ README; câu diễn giải là gợi ý điều hướng của người viết. [R1]

Platforms	Chọn hệ sinh thái và môi trường chạy. Ví dụ: Node.js
Programming Languages	Tìm tài nguyên theo ngôn ngữ đang học. Ví dụ: JavaScript
Front-End Development	Xây giao diện và trải nghiệm trên trình duyệt. Ví dụ: ES6 Tools
Back-End Development	Tổ chức dịch vụ và xử lý phía máy chủ. Ví dụ: Flask
Computer Science	Tìm nền tảng khoa học máy tính. Ví dụ: University Courses
Big Data	Khám phá hệ thống và nguồn dữ liệu lớn. Ví dụ: Big Data
Theory	Đọc lý thuyết, bài nghiên cứu và ý tưởng nền. Ví dụ: Papers We Love
Books	Chọn sách theo nhu cầu và kiến thức nền. Ví dụ: Free Programming Books
Editors	Tìm trình soạn thảo và cách sử dụng. Ví dụ: Sublime Text
Gaming	Tìm tài nguyên làm hoặc nghiên cứu game. Ví dụ: Game Development
Development Environment	Chuẩn bị môi trường và quy trình phát triển. Ví dụ: Quick Look Plugins
Entertainment	Khám phá các chủ đề giải trí. Ví dụ: Science Fiction
Databases	Tìm công nghệ lưu trữ và truy vấn dữ liệu. Ví dụ: Database
Media	Tìm công cụ, định dạng và tài nguyên truyền thông. Ví dụ: Fonts

Bản đồ 27 nhóm / 02

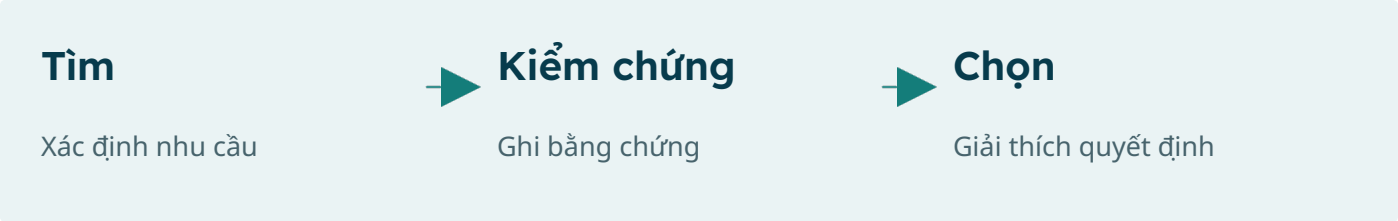
Chọn nhóm theo nhiệm vụ. Ví dụ lấy từ README; câu diễn giải là gợi ý điều hướng của người viết. [R1]

Learn	Chọn cách học và bài thực hành. Ví dụ: CLI Workshoppers
Security	Tìm kiến thức và công cụ bảo mật đúng phạm vi. Ví dụ: Application Security
Content Management Systems	Tìm hệ quản trị nội dung. Ví dụ: Umbraco
Hardware	Khám phá tài nguyên phần cứng và thiết bị. Ví dụ: Robotics
Business	Tìm tài nguyên về tổ chức và kinh doanh. Ví dụ: Open Companies
Work	Tìm tài nguyên phục vụ công việc. Ví dụ: Slack
Networking	Khám phá công cụ và kiến thức mạng. Ví dụ: PCAPTools
Decentralized Systems	Tìm tài nguyên về hệ thống phi tập trung. Ví dụ: Bitcoin
Health and Social Science	Tìm nguồn về sức khỏe và khoa học xã hội. Ví dụ: Biomedical Information Extraction
Events	Tìm sự kiện phù hợp chủ đề và địa điểm. Ví dụ: Creative Tech Events
Testing	Tìm phương pháp và công cụ kiểm thử. Ví dụ: Testing
Miscellaneous	Khám phá các chủ đề không nằm ở nhóm khác. Ví dụ: Scientific Writing
Related	Tìm cách truy cập và khám phá hệ Awesome. Ví dụ: All Awesome Lists

Hiểu đúng Awesome

Kho này thực sự giúp mình làm việc gì?

Awesome tổ chức việc khám phá tài nguyên theo hai tầng: danh mục tổng dẫn tới danh sách chuyên ngành; danh sách chuyên ngành dẫn tới công cụ, tài liệu và dự án. Giá trị cốt lõi là tuyển chọn có lý do. Một liên kết xuất hiện ở đây là điểm khởi đầu để tìm hiểu, không phải lời bảo đảm phù hợp với mọi người. Chương này giúp bạn chuyển một mong muốn mơ hồ thành nhiệm vụ có thể hoàn thành và biết xác định mình đang ở tầng nào của hệ thống. [R1][R2]



BA KHÁI NIỆM CẦN GIỮ LẠI

Repository	Kho lưu trữ nội dung, mã nguồn và lịch sử thay đổi.
Curation	Tuyển chọn có chủ đích, có lý do và phạm vi.
README	Trang giới thiệu và điều hướng chính của một repo.

Đặt câu hỏi trước khi mở danh sách

MỤC TIÊU / Viết được một yêu cầu có đầu ra và ràng buộc rõ ràng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R2] ↗

HIỂU CƠ CHẾ

Nếu bạn chỉ hỏi "có công cụ nào hay?", danh sách càng lớn càng dễ làm bạn phân tâm. Hãy bắt đầu bằng một việc muốn làm, ai sử dụng, điều kiện hiện tại và dấu hiệu thành công. Ví dụ của người viết: "Tôi muốn chọn một tài liệu nhập môn JavaScript có bài tập, dùng trên máy đang có, rồi tự giải thích được ví dụ đầu tiên". Câu hỏi này cho phép loại tài nguyên ngoài phạm vi. Manifesto yêu cầu danh sách có phạm vi rõ; người đọc cũng cần một phạm vi rõ để dùng danh sách hiệu quả. [R2]

LÀM THEO TỪNG BƯỚC

1. Mở mẫu project-brief.md trong bộ bài tập hoặc tạo một ghi chú trống. Viết một câu về việc muốn hoàn thành, tránh dùng "học mọi thứ".
2. Ghi kiến thức đang có, thiết bị, ngôn ngữ tài liệu đọc được và chi phí chấp nhận. Những gì chưa biết thì ghi "chưa xác định".
3. Định nghĩa một đầu ra kiểm chứng được: một trang ghi chú, một ví dụ chạy được hoặc bảng so sánh có bằng chứng.
4. Viết hai điều chủ động không làm. Ví dụ: chưa chọn framework, chưa tối ưu hiệu năng khi vẫn đang học cú pháp.

ĐẦU RA CẦN ĐẠT

Một bản yêu cầu 4 mục: mục tiêu, ràng buộc, đầu ra, ngoài phạm vi.

LỖI THƯỜNG GẶP

Đặt mục tiêu bằng tên công nghệ như "học AI" nhưng không có tác vụ. Sửa bằng một sản phẩm nhỏ hoặc một câu hỏi có thể trả lời.

TỰ KIỂM TRA

Tại sao "tìm thư viện nhiều sao nhất" chưa đủ làm yêu cầu?

Gợi ý: Vì nó không xác định nhiệm vụ, ràng buộc hay cách nhận biết thư viện phù hợp.

Làm thêm: Viết lại mục tiêu của bạn thành một câu có động từ và đầu ra. Nhờ một người khác đọc và nói lại điều họ hiểu.

moda.app

Đi qua đúng ba tầng liên kết

MỤC TIÊU / Phân biệt repo tổng, danh sách chuyên ngành và tài nguyên đích.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R1] ↗

HIỂU CƠ CHẾ

Ở repo tổng, một mục thường là liên kết tới một danh sách khác, không phải gói phần mềm để cài. Ví dụ thật: Platforms có Node.js và trỏ tới awesome-nodejs. Sau khi mở danh sách chuyên ngành, bạn mới chọn mục liên quan rồi đọc tài liệu của tài nguyên đó. Quy tắc đóng góp cũng phân biệt thêm một danh sách vào repo tổng với thêm một tài nguyên vào danh sách đang có. Nhầm hai tầng này dễ khiến bạn gửi thay đổi sai nơi hoặc tưởng mình đã đọc tài liệu chính thức trong khi mới đọc một câu giới thiệu. [R1][R3]

LÀM THEO TỪNG BƯỚC

1. Mở readme.md của sindresorhus/awesome, tìm Contents và chọn Platforms.
2. Mở mục Node.js. Ghi URL danh sách awesome-nodejs, rồi tìm phần Contents của danh sách đó.
3. Chọn một mục gần yêu cầu của bài 1. Mở liên kết đích và xác định trang giới thiệu hoặc tài liệu chính thức của nó.
4. Ghi lại ba dòng: repo tổng → danh sách chuyên ngành → tài nguyên đích. Viết một câu mô tả vai trò của từng dòng.

ĐẦU RA CẦN ĐẠT

Một đường đi gồm ba URL thật, kèm vai trò của từng URL.

LỖI THƯỜNG GẶP

Tải repo tổng rồi mong có toàn bộ tài liệu bên ngoài. Repo tổng chỉ lưu nội dung của chính nó và các liên kết.

TỰ KIỂM TRA

Muốn đề xuất một thư viện Node.js, bạn thường đọc hướng dẫn đóng góp ở đâu?

Gợi ý: Ở danh sách Node.js đích; repo tổng chủ yếu tiếp nhận danh sách theo chủ đề.

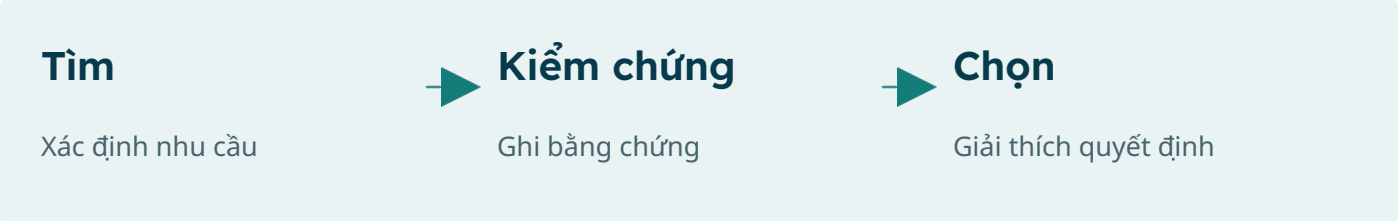
Làm thêm: Lặp lại đường đi với Python trong Programming Languages. So sánh hai danh sách theo cách chia mục, chưa kết luận chất lượng thư viện.

moda.app

Đọc bản đồ 27 nhóm

Chọn lối vào nào cho câu hỏi của mình?

Các nhóm trong README là các lối vào khám phá, không phải thứ tự môn học. Platforms tổ chức quanh môi trường chạy; Programming Languages quanh ngôn ngữ; Front-End và Back-End quanh vai trò ứng dụng. Những nhóm như Theory, Books và Learn tổ chức quanh dạng tri thức hoặc cách học. Một tài nguyên có thể liên quan tới nhiều lối vào. Vì vậy, chọn nhóm theo nhiệm vụ rồi đối chiếu phạm vi danh sách đích sẽ hữu ích hơn việc cố tìm một nhãn duy nhất đúng tuyệt đối. [R1]



BA KHÁI NIỆM CẦN GIỮ LẠI

Platform	Nền tảng hoặc môi trường mà ứng dụng hoạt động.
Taxonomy	Hệ phân loại giúp chia nội dung thành nhóm.
Learning path	Lộ trình học do bạn xây từ các tài nguyên.

Chọn nhóm theo việc cần làm

MỤC TIÊU / Tìm được hai lối vào hợp lý và giải thích được sự khác nhau.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R1] ↗

HIỂU CƠ CHẾ

Muốn làm web có thể bắt đầu từ Front-End Development, Back-End Development hoặc một ngôn ngữ quen thuộc. Muốn hiểu nền tảng thì Computer Science, Theory và Books là những cửa vào khác. Không nên coi các nhóm này là mức độ từ dễ tới khó: README không đưa ra một chương trình học bắt buộc. Hai trang bản đồ ở đầu tài liệu liệt kê đủ 27 nhóm cùng một liên kết mẫu để bạn thử. Các câu hỏi và bài tập gắn với mỗi nhóm là cách biên soạn của người viết, không phải lời hứa rằng danh sách đích sẽ dạy đầy đủ chủ đề đó. [R1]

LÀM THEO TỪNG BƯỚC

1. Lấy yêu cầu bài 1 và khoanh hai nhóm có liên quan trong bản đồ 27 nhóm.
2. Đọc mô tả của một mục trong mỗi nhóm, rồi mở README của hai danh sách đích.
3. Viết phạm vi mỗi danh sách bằng một câu. Nếu không thấy phạm vi rõ, ghi đó là điểm chưa xác minh.
4. Chọn một đường đi chính và một đường dự phòng. Ghi lý do dựa vào mục tiêu, không dựa duy nhất vào số sao.

ĐẦU RA CẦN ĐẠT

Hai đường điều hướng và một quyết định có lý do.

LỖI THƯỜNG GẶP

Cho rằng nhóm đứng trước trong README là kiến thức cần học trước. Thứ tự mục lục là điều hướng, không tự động là thứ tự học.

TỰ KIỂM TRA

Vì sao một nhu cầu có thể có hai đường đi đúng?

Gợi ý: Vì danh mục chia theo nhiều góc nhìn: môi trường, vai trò công việc, lĩnh vực hoặc dạng tài liệu.

Làm thêm: Làm cùng bài tập với một nhu cầu ngoài lập trình, chẳng hạn tìm tài nguyên làm việc từ xa hoặc viết khoa học.

moda.app

Biến danh sách thành lộ trình học

MỤC TIÊU / Sắp được ba bước học có quan hệ đầu vào–đầu ra.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R1] ↗

HIỂU CƠ CHẾ

Một danh sách phong phú không tự tạo thành giáo trình. Bạn cần tự bổ sung thứ tự, kiến thức nền và bài kiểm tra. Đề xuất của người viết là chia thành ba bước: hiểu khái niệm, làm một tác vụ nhỏ, rồi mở rộng hoặc so sánh. Với mỗi bước, chỉ chọn một tài nguyên chính để giảm chuyển đổi liên tục. Ví dụ: đọc tài liệu nhập môn trước khi dùng một công cụ nâng cao. Các nhóm Learn, Books và Computer Science trong README hỗ trợ việc khám phá nguồn; thứ tự học và thời gian phải được điều chỉnh theo người học. [R1]

LÀM THEO TỪNG BƯỚC

1. Chọn ba câu hỏi theo thứ tự: cần hiểu gì, cần làm được gì, cần tự thay đổi điều gì.
2. Gắn một tài nguyên chính vào mỗi câu hỏi và ghi kiến thức nền mà tài nguyên đó yêu cầu.
3. Đặt đầu ra cuối từng bước. Chỉ chuyển bước khi bạn có đầu ra, thay vì chỉ đánh dấu đã mở trang.
4. Nếu một tài nguyên quá khó, quay lại một bước nền tảng. Ghi cụ thể thuật ngữ hoặc thao tác khiến bạn mắc kẹt.

ĐẦU RA CẦN ĐẠT

Lộ trình ba bước, mỗi bước có tài nguyên, điều kiện bắt đầu và đầu ra.

LỖI THƯỜNG GẶP

Lập một danh sách nhiều khóa học nhưng không có bài thực hành. Sửa bằng một đầu ra nhỏ cho mỗi nguồn chính.

TỰ KIỂM TRA

Khi nào nên đổi tài nguyên?

Gợi ý: Khi có bằng chứng nó lệch mục tiêu hoặc thiếu kiến thức nền; không đổi chỉ vì vừa thấy một liên kết mới hấp dẫn.

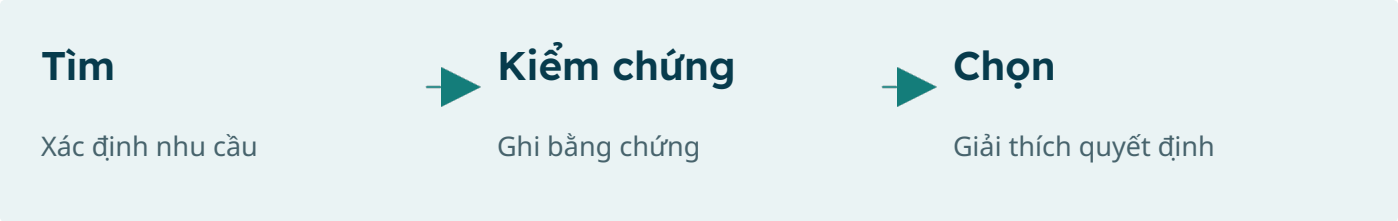
Làm thêm: Chọn một bước và hoàn thành đầu ra trong phạm vi bạn có thể quản lý. Ghi lại điều chưa hiểu thay vì tăng thêm số tài nguyên.

moda.app

Tuyển chọn bằng bằng chứng

Tại sao một tài nguyên xứng đáng xuất hiện?

The awesome manifesto yêu cầu chỉ giới thiệu thứ bạn hoặc người đóng góp có thể thật sự khuyên dùng, và giải thích lợi ích của nó. Hướng dẫn danh sách còn loại tài nguyên không được duy trì, đã lưu trữ, ngừng dùng hoặc thiếu tài liệu khỏi danh sách chính; trường hợp cần giữ nên nằm trong tệp riêng. Chương này chuyển tinh thần ấy thành hồ sơ bằng chứng và bảng so sánh. Các thang điểm trong bài là công cụ ra quyết định do người viết đề xuất, không phải công thức chính thức của Awesome. [R2][R5]



BA KHÁI NIỆM CẦN GIỮ LẠI

Evidence	Bằng chứng có thể mở lại hoặc tái kiểm tra.
Archived	Repo được chủ dự án chuyển sang trạng thái lưu trữ.
Trade-off	Điểm đánh đổi giữa các lựa chọn.

Lập hồ sơ ứng viên

MỤC TIÊU / Phân biệt điều đã quan sát với điều đang suy đoán.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R2] ↗

HIỂU CƠ CHẾ

Một mô tả hấp dẫn chưa đủ để khuyên người khác dùng. Hãy ghi URL chính, phạm vi, người dùng phù hợp, tài liệu bắt đầu, giấy phép và tình trạng duy trì quan sát được. Mọi nhận xét cần có ngày kiểm tra và bằng chứng. Ví dụ "đã lưu trữ" phải dựa trên trạng thái repo; "dễ dùng" cần một tác vụ bạn đã thử. Không có commit gần đây chưa đủ để kết luận một dự án ngừng hoạt động, bởi một tài nguyên ổn định có thể ít thay đổi. Hướng dẫn Awesome yêu cầu chất lượng tuyển chọn; hồ sơ này là cách thực hành của người viết. [R2][R5]

LÀM THEO TỪNG BƯỚC

1. Mở mẫu candidate-log.csv. Tạo một dòng cho mỗi ứng viên, ghi ngày, URL và nhiệm vụ dự định thử.
2. Mở tài liệu gốc và giấy phép. Chép liên kết bằng chứng; mục không tìm thấy phải ghi "chưa xác minh".
3. Kiểm tra trạng thái lưu trữ, thông báo ngừng dùng và hướng dẫn bắt đầu. Tách quan sát khỏi diễn giải cá nhân.
4. Thử một tác vụ phù hợp với phạm vi tài nguyên. Ghi đầu vào, việc làm, kết quả và điều còn hạn chế.

ĐẦU RA CẦN ĐẠT

Hai hồ sơ ứng viên có liên kết bằng chứng và trạng thái đã thử/chưa thử.

LỖI THƯỜNG GẶP

Điền "miễn phí, an toàn, dễ dùng" theo ấn tượng. Sửa bằng điều kiện cụ thể, nguồn gốc và kết quả thử thực tế.

TỰ KIỂM TRA

Bạn chưa tìm thấy giấy phép thì nên ghi gì?

Gợi ý: Ghi chưa xác minh, rồi tìm thêm hoặc hỏi chủ dự án; không tự suy ra quyền sử dụng.

Làm thêm: Chọn một nhận xét của bạn và thử bác bỏ nó bằng cách tìm trường hợp không còn đúng.

So sánh và loại có lý do

MỤC TIÊU / Ra quyết định có thể giải thích lại cho người khác.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R2] ↗

HIỂU CƠ CHẾ

Khi có nhiều ứng viên, hãy dùng cùng một tập tiêu chí. Đề xuất của người viết gồm: phù hợp nhiệm vụ, rõ tài liệu, kết quả thử và điều kiện sử dụng. Bạn có thể chấm mỗi tiêu chí từ 0 đến 2, nhưng điểm tổng không thay thế điều kiện bắt buộc. Nếu dự án của bạn cần quyền sử dụng nhất định mà chưa xác minh được, đó là điểm chặn cho quyết định hiện tại, dù những tiêu chí khác cao. Manifesto khuyến khích bỏ bớt thay vì thêm quá nhiều; mục tiêu của so sánh là lựa chọn có lý do, không phải tạo bảng xếp hạng phổ quát. [R2]

LÀM THEO TỪNG BƯỚC

1. Chọn hai ứng viên cho cùng một nhiệm vụ. Viết trước các điều kiện bắt buộc, tránh đổi tiêu chí để bên mình thích thắng.
2. Chấm bốn tiêu chí theo bằng chứng: 0 chưa đạt/chưa rõ, 1 đạt một phần, 2 đạt tác vụ đã xác định.
3. Ghi một ưu điểm, một hạn chế và một tình huống không phù hợp cho từng ứng viên.
4. Chọn một, hoãn một hoặc loại cả hai. Viết đoạn quyết định nêu điều kiện khiến bạn sẽ xem xét lại.

ĐẦU RA CẦN ĐẠT

Một bảng so sánh và đoạn kết luận khoảng 5 câu.

LỖI THƯỜNG GẶP

Cộng điểm rồi bỏ qua điều kiện bắt buộc. Luôn kiểm tra điểm chặn trước khi dùng điểm tổng để hỗ trợ so sánh.

TỰ KIỂM TRA

Hai ứng viên cùng điểm thì làm gì?

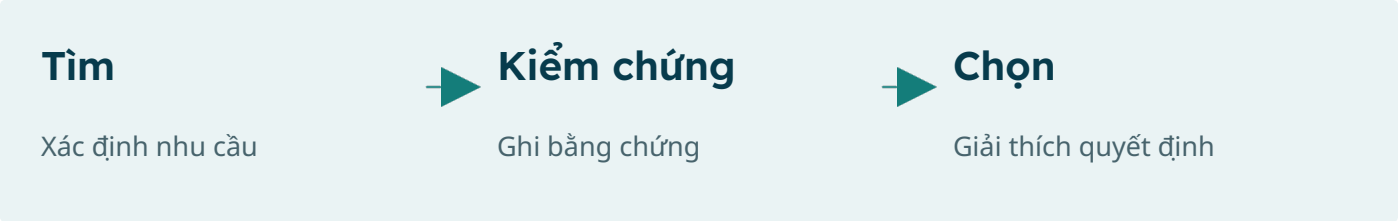
Gợi ý: Thử tác vụ phân biệt quan trọng nhất hoặc chọn theo ràng buộc của người dùng; không bịa độ chính xác cho điểm số.

Làm thêm: Đưa bảng cho người khác và hỏi họ có thể xác minh từng ô từ liên kết bạn ghi hay không.

Viết mục giới thiệu có ích

Người đọc hiểu được gì sau một dòng mô tả?

Awesome yêu cầu mô tả ngắn gọn nhưng có thông tin. Cần phân biệt câu mô tả một tài nguyên với câu mô tả chủ đề của một danh sách khi đề xuất vào repo tổng. Hướng dẫn repo tổng yêu cầu mô tả dự án/chủ đề, không dùng khẩu hiệu và không chỉ nói "một danh sách tài nguyên". Các mục nên nhất quán về cách viết tên, dấu phân cách và dấu chấm. Chương này giúp bạn đi từ ghi chú quan sát dài tới một mô tả ngắn, rồi thử xem người khác có hiểu đúng không. [R2][R5]



BA KHÁI NIỆM CẦN GIỮ LẠI

Entry	Một mục gồm tên, liên kết và mô tả.
Description	Câu nêu chức năng hoặc phạm vi cụ thể.
Editorial review	Đọc lại để sửa nghĩa, cấu trúc và cách diễn đạt.

Viết một entry không quảng cáo

MỤC TIÊU / Viết được mô tả chức năng có thể kiểm chứng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R5] ↗

HIỂU CƠ CHẾ

Một entry có ích cho biết tài nguyên làm gì và giúp người đọc quyết định có mở tiếp không. Các từ như "tốt nhất", "cực mạnh" thường không cung cấp điều kiện để đánh giá. Theo hướng dẫn, tên và mô tả được ngăn bằng dấu gạch ngang; mô tả bắt đầu bằng chữ hoa và kết thúc bằng dấu chấm. Khi đề xuất một danh sách lên repo tổng, câu mô tả nói về chủ đề của danh sách, không quảng cáo chính danh sách. Ví dụ iOS trong hướng dẫn mô tả hệ điều hành, thay vì nói "tài nguyên và công cụ cho iOS". [R5]

LÀM THEO TỪNG BƯỚC

1. Chọn một ứng viên đã có hồ sơ bằng chứng. Viết một câu trả lời "nó cung cấp chức năng gì?".
2. Xóa từ quảng cáo, nhận xét chưa thử và thông tin không liên quan tới nhiệm vụ người đọc.
3. Giữ đúng tên dự án, URL gốc và mô tả. Dùng đoạn mẫu entry trong bộ bài tập để hiểu cú pháp Markdown.
4. Đọc lại câu với tài liệu gốc. Nếu câu vượt quá điều bạn đã kiểm chứng, thu hẹp nó hoặc bổ sung bằng chứng.

ĐẦU RA CẦN ĐẠT

Hai entry cùng định dạng và mỗi entry có lý do chọn nằm trong nhật ký riêng.

LỖI THƯỜNG GẶP

Đưa lời khen vào mô tả nhưng bỏ chức năng. Sửa bằng danh từ và động từ cụ thể có trong tài liệu gốc.

TỰ KIỂM TRA

Mô tả ngắn có cần kể hết tính năng không?

Gợi ý: Không. Nó cần nêu phần chức năng hoặc phạm vi đủ để người đọc sàng lọc; chi tiết nằm ở tài nguyên đích.

Làm thêm: Nhờ người khác nói lại ứng viên làm gì chỉ từ entry; sửa câu nếu họ hiểu lệch.

moda.app

Thử trải nghiệm của người đọc

MỤC TIÊU / Đo xem danh sách có giúp người mới tìm được thứ cần không.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R2] ↗

HIỂU CƠ CHẾ

Tuyển chọn không dừng ở kiểm tra tài nguyên. Một danh sách có mục chất lượng vẫn khó dùng nếu nhóm mơ hồ, mô tả trùng nhau hoặc không có mục lục. Manifesto yêu cầu tổ chức theo nhóm và giữ định dạng nhất quán. Bài tập của người viết là thử một nhiệm vụ tìm kiếm: đưa người đọc một câu hỏi, để họ tự chọn đường đi rồi giải thích lựa chọn. Không cần đặt một giới hạn thời gian chung cho mọi người; điều cần ghi là vị trí họ nhầm và thông tin họ thiếu. Kết quả giúp sửa cấu trúc thay vì chỉ thêm liên kết. [R2]

LÀM THEO TỪNG BƯỚC

1. Chọn một câu hỏi có thể trả lời bằng các entry hiện có. Không chỉ sẵn mục cần mở.
2. Nhờ một người đọc thử, hoặc tự đọc lại sau khi tạm rời tài liệu. Ghi điểm họ dừng hoặc hiểu nhầm.
3. Hỏi vì sao họ chọn entry đó, chức năng họ mong đợi và điều kiện họ chưa biết.
4. Sửa một nhãn nhóm hoặc mô tả, rồi thử lại đúng nhiệm vụ để xem sự nhầm lẫn có giảm không.

ĐẦU RA CẦN ĐẠT

Một ghi chú thử đọc gồm nhiệm vụ, quan sát, thay đổi và kết quả lần thử sau.

LỖI THƯỜNG GẶP

Đánh giá bằng câu “giao diện đẹp” mà không biết người đọc tìm được gì. Hãy quan sát một nhiệm vụ cụ thể.

TỰ KIỂM TRA

Người đọc chọn sai có luôn là lỗi của họ không?

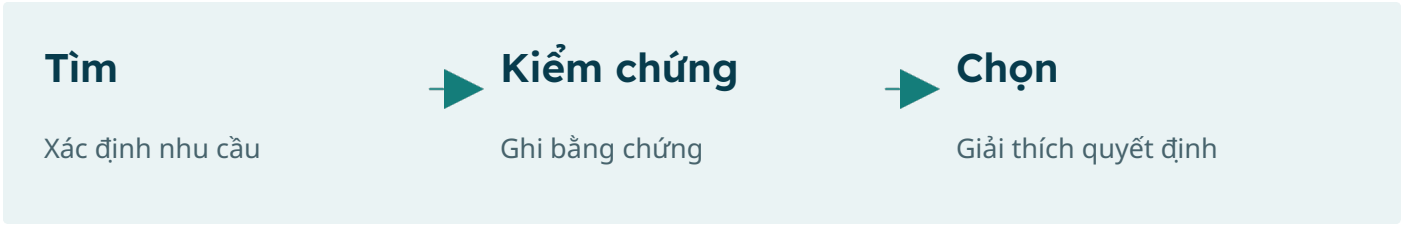
Gợi ý: Không. Có thể nhãn, mô tả hoặc cấu trúc tạo ra kỳ vọng sai; cần xem lại bằng chứng quan sát.

Làm thêm: Đổi một entry sang nhóm khác và giải thích vì sao vị trí mới hợp với nhiệm vụ hơn.

Tự xây danh sách của mình

Một repo tuyển chọn cần những thành phần nào?

Danh sách mới cần có chủ đề rõ, tránh trùng một danh sách tốt đã tồn tại, và do người duy trì thật sự đầu tư công sức. Hướng dẫn hiện hành nêu tên repo dạng awesome-name-of-list, nhánh mặc định main, tiêu đề theo title case, mục lục Contents và hướng dẫn đóng góp. Quy tắc giấy phép khuyến nghị CC0 hoặc Creative Commons phù hợp; đây là giấy phép của danh sách, không tự thay đổi giấy phép của các dự án được liên kết. Bộ khung trong giáo trình là bài tập, chưa phải danh sách đủ điều kiện gửi lên Awesome. [R4][R5]



BA KHÁI NIỆM CẦN GIỮ LẠI

Scope	Ranh giới chủ đề và đối tượng của danh sách.
License	Điều kiện cho phép sử dụng nội dung.
Contribution guide	Hướng dẫn người khác đề xuất thay đổi.

Tạo README có phạm vi và mục lục

MỤC TIÊU / Tạo được bản nháp danh sách có cấu trúc rõ ràng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R4] ↗

HIỂU CƠ CHẾ

Trước khi tạo danh sách mới, hãy tìm danh sách tương tự. create-list.md khuyên đóng góp vào danh sách tốt đã có nếu chủ đề trùng. Nếu nhu cầu thật sự khác, bạn cần mô tả sự khác biệt bằng phạm vi hoặc đối tượng, không chỉ đổi tên. Mục lục Contents là phần đầu tiên của danh sách; hướng dẫn yêu cầu ít lồng cấp và không đưa Contributing hoặc Footnotes vào mục lục. Người học có thể tạo file trên GitHub bằng trình duyệt, chưa cần Git hay Node.js. Bộ khung kèm theo chỉ chỉ ra cấu trúc để bạn tự viết nội dung. [R4][R5]

LÀM THEO TỪNG BƯỚC

1. Tìm trong repo tổng và các đề xuất đang có; ghi danh sách tương tự cùng lý do phạm vi của bạn khác.
2. Tạo repo bài tập của riêng bạn. Nếu định theo chuẩn Awesome, dùng tên dạng awesome-ten-chu-de và nhánh main.
3. Tạo README với tiêu đề, mô tả chủ đề, Contents và một vài nhóm đủ rõ để phân loại ứng viên.
4. Tự thêm các entry đã kiểm chứng; dùng Preview để thử mục lục và liên kết. Không sao chép cả danh sách khác rồi nhận là tuyển chọn của mình.

ĐẦU RA CẦN ĐẠT

README nháp có phạm vi, mục lục và các entry do bạn tự đánh giá.

LỖI THƯỜNG GẶP

Tạo quá nhiều nhóm rỗng hoặc một chủ đề quá rộng. Giảm phạm vi cho tới khi bạn giải thích được tiêu chí nhận và loại.

TỰ KIỂM TRA

Có cần tạo một repo mới cho mọi chủ đề không?

Gợi ý: Không. Nếu đã có danh sách tốt cùng phạm vi, đóng góp vào đó có thể hữu ích hơn tạo bản trùng.

Làm thêm: Viết một câu “danh sách này dành cho...” và một câu “danh sách này không bao gồm...”.

Thêm quy tắc đóng góp và giấy phép

MỤC TIÊU / Giúp người khác hiểu cách đóng góp và quyền sử dụng nội dung.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R5] ↗

HIỂU CƠ CHẾ

Người đóng góp cần biết mục nào được chấp nhận, bằng chứng phải có và cách mô tả thay đổi. Manifesto cho phép tham khảo contributing.md của repo để điều chỉnh. Với giấy phép, hướng dẫn Awesome khuyến nghị mạnh CC0 và chấp nhận Creative Commons; các giấy phép mã nguồn như MIT hoặc GPL không phù hợp với yêu cầu danh sách của repo tổng. Điều này nói về nội dung danh sách. Một thư viện được liên kết vẫn giữ giấy phép riêng của nó. Nếu bạn chưa rõ quyền với hình ảnh hoặc phần sao chép, hãy kiểm tra nguồn thay vì gán giấy phép cho mọi thứ. [R2][R5]

LÀM THEO TỪNG BƯỚC

1. Tạo contributing.md với phạm vi, tiêu chí chất lượng, định dạng entry, cách kiểm tra trùng và yêu cầu nêu lý do đề xuất.
2. Chọn giấy phép theo ý định sử dụng và hướng dẫn Awesome. Dùng nội dung giấy phép chuẩn từ nguồn chính thức, không tự viết lại điều khoản.
3. Đặt tệp LICENSE hoặc license ở gốc repo theo hướng dẫn. Kiểm tra riêng quyền của hình ảnh hoặc nội dung bên thứ ba nếu có.
4. Kiểm tra badge và cách đặt tiêu đề theo hướng dẫn; đừng dùng badge để tuyên bố danh sách đã được repo tổng chấp nhận.

ĐẦU RA CẦN ĐẠT

Một contributing.md rõ tiêu chí và một giấy phép đã được bạn đọc, lựa chọn.

LỖI THƯỜNG GẶP

Cho rằng giấy phép danh sách cấp quyền dùng mọi dự án được dẫn link. Quyền của mỗi dự án cần được kiểm tra riêng.

TỰ KIỂM TRA

Badge Awesome có chứng minh danh sách đã được nhận vào repo tổng không?

Gợi ý: Không. Manifesto cho phép dùng badge cả trên danh sách chưa có trong repo tổng.

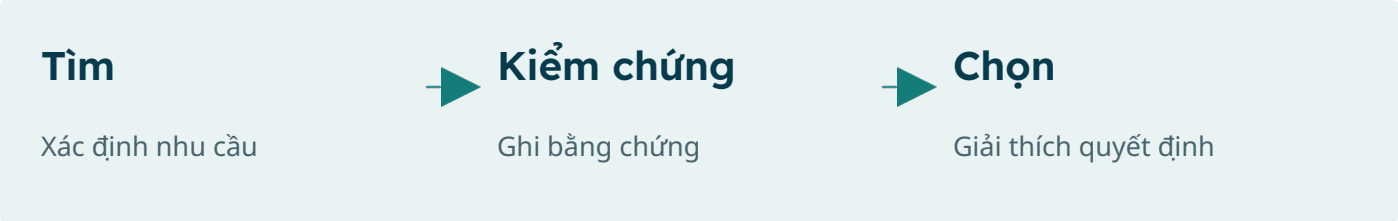
Làm thêm: Đưa quy tắc đóng góp cho người khác và hỏi họ có biết phải nộp bằng chứng gì cho một entry mới không.

moda.app

Kiểm tra chất lượng

Làm sao biết danh sách đã sẵn sàng để người khác đọc?

Kiểm tra có hai lớp: chất lượng nội dung và tính nhất quán định dạng. Người duy trì phải đánh giá phạm vi, bằng chứng, tài liệu và trạng thái dự án; công cụ có thể hỗ trợ phát hiện lỗi Markdown và quy tắc Awesome. awesome-lint là công cụ riêng, yêu cầu Node.js và Git khi dùng CLI. Một báo cáo lint sạch không chứng minh mọi tài nguyên tốt, cũng không tự vượt các yêu cầu tuổi danh sách và đầu tư thực tế. [R5][R6]



BA KHÁI NIỆM CẦN GIỮ LẠI

Lint	Kiểm tra tự động các quy tắc định dạng hoặc cấu trúc.
False positive	Cảnh báo công cụ đưa ra nhưng không phản ánh lỗi thực tế.
Quality gate	Điều kiện cần đạt trước khi chuyển bước.

Rà soát thủ công trước khi chạy công cụ

MỤC TIÊU / Phát hiện trùng lặp, entry lệch phạm vi và bằng chứng còn thiếu.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R5] ↗

HIỂU CƠ CHẾ

Một máy kiểm tra có thể tìm lỗi dấu câu nhưng không tự hiểu đầy đủ liệu tài nguyên có đáng được giới thiệu cho người đọc của bạn hay không. Hãy bắt đầu bằng phạm vi và hồ sơ ứng viên: mỗi mục có chức năng rõ, có lý do chọn và còn phù hợp không? Hướng dẫn yêu cầu không chứa tài nguyên đã lưu trữ, ngừng dùng, thiếu tài liệu hoặc không được duy trì trong danh sách chính. Những tài nguyên lịch sử thật sự cần giữ có thể chuyển sang tệp riêng với giải thích. Đừng dùng một con số về hoạt động commit làm tiêu chuẩn tự động duy nhất. [R2][R5]

LÀM THEO TỪNG BƯỚC

1. Đọc toàn bộ entry và so với phạm vi đã viết. Đánh dấu mục không giải thích được lý do tồn tại.
2. Mở từng liên kết chính, kiểm tra trùng URL hoặc hai tên cùng chỉ một tài nguyên.
3. Đối chiếu tài liệu, trạng thái repo và nhật ký bằng chứng; ghi ngày kiểm tra để lần sau biết điều cần xem lại.
4. Thử mục lục ở chế độ Preview, sửa nhãn không rõ và bảo đảm Contributing, Footnotes không xuất hiện trong Contents theo yêu cầu.

ĐẦU RA CẦN ĐẠT

Checklist có trạng thái đạt/chưa đạt và việc cần sửa cho từng lỗi.

LỖI THƯỜNG GẶP

Xóa một mục chỉ vì lâu không có commit. Tìm thêm thông báo, tình trạng tài liệu và kết quả sử dụng trước khi kết luận.

TỰ KIỂM TRA

Liên kết mở được có đủ chứng minh entry tốt không?

Gợi ý: Không. Nó chỉ xác nhận truy cập; vẫn cần kiểm tra nội dung, phạm vi và khả năng sử dụng.

Làm thêm: Tìm một entry của chính bạn có bằng chứng yếu nhất và cải thiện hồ sơ trước khi thêm entry mới.

Chạy awesome-lint và hiểu kết quả

MỤC TIÊU / Dùng được công cụ kiểm tra mà không nhầm lint với đánh giá chất lượng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R6] ↗

HIỂU CƠ CHẾ

README awesome-lint mô tả cách kiểm tra một repo từ URL và cách chạy trong thư mục hiện tại. CLI cần Node.js và Git. Lệnh có thể tải gói để thực thi, vì vậy người học cần xem tài liệu của công cụ trước khi chạy. Ví dụ kết quả trong README gồm thiếu badge, kiểu dấu đầu dòng và dấu gạch chéo cuối URL. Hãy đọc tên quy tắc, vị trí lỗi rồi sửa đúng nguyên nhân. Repo tổng yêu cầu chạy công cụ và xử lý lỗi; nếu có cảnh báo sai, hướng dẫn yêu cầu báo vấn đề thay vì che hết cảnh báo cho qua. [R5][R6]

LÀM THEO TỪNG BƯỚC

1. Chọn cách làm phù hợp. Nếu chưa có Node.js và Git, hoàn thành bài rà soát thủ công và đánh dấu lint chưa chạy.
2. Khi có công cụ, mở terminal tại bản sao repo của bạn. Lệnh nguồn dùng trong thư mục hiện tại là npx awesome-lint.
3. Đọc từng thông báo theo tệp, dòng, cột và tên quy tắc; sửa một nhóm lỗi rồi chạy lại.
4. Lưu kết quả vào nhật ký. Nếu nghi cảnh báo sai, ghi ví dụ tối thiểu và đối chiếu tài liệu trước khi đề xuất thay đổi quy tắc.

ĐẦU RA CẦN ĐẠT

Một nhật ký lint nêu môi trường, lệnh, lỗi trước/sau hoặc lý do chưa thể chạy.

LỖI THƯỜNG GẶP

Tắt toàn bộ quy tắc để có kết quả sạch. Báo cáo sạch chỉ có ý nghĩa khi các kiểm tra phù hợp vẫn hoạt động.

TỰ KIỂM TRA

Lint sạch có làm danh sách đủ điều kiện gửi lên repo tổng không?

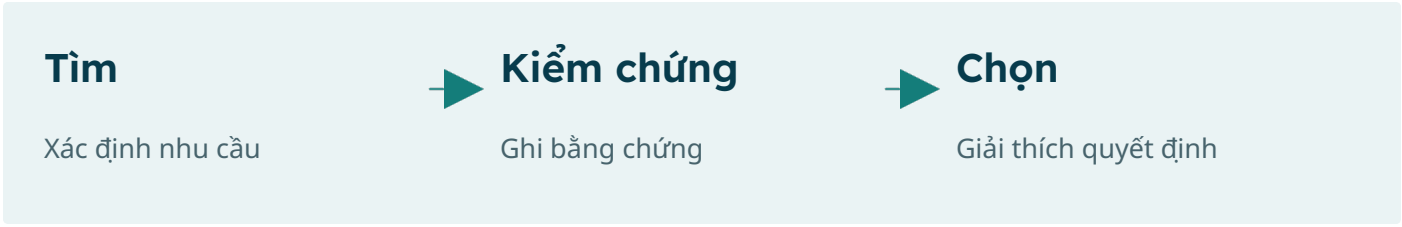
Gợi ý: Không. Còn các điều kiện về chất lượng, tuổi danh sách, chính sách nội dung và đóng góp cộng đồng.

Làm thêm: Giải thích một lỗi lint bằng ngôn ngữ của bạn, rồi chỉ ra thay đổi sửa nguyên nhân của nó.

Đóng góp đúng nơi, đúng điều kiện

Khi nào và bằng cách nào nên đề xuất thay đổi?

Có hai luồng khác nhau: đề xuất một tài nguyên vào danh sách chuyên ngành và đề xuất cả một danh sách vào repo tổng. Hướng dẫn trình bày thao tác chỉnh README qua trình duyệt rồi gửi PR. Điều kiện repo tổng chặt hơn: tuổi ít nhất 30 ngày, không phải danh sách hay PR hoàn toàn do AI tạo, và yêu cầu review có nội dung cho ít nhất bốn PR khác. Phải đọc trạng thái tiếp nhận hiện tại; làm xong một bài tập không tự tạo quyền gửi thay đổi vào cộng đồng. [R3][R4][R5]



BA KHÁI NIỆM CẦN GIỮ LẠI

Pull request (PR)	Đề xuất thay đổi để người duy trì xem xét.
Maintainer	Người duy trì dự án và quy tắc tiếp nhận.
Review	Đọc, kiểm chứng và phản hồi cụ thể về thay đổi.

Đề xuất một tài nguyên vào danh sách có sẵn

MỤC TIÊU / Soạn được một thay đổi nhỏ, đúng phạm vi và có lý do.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R3] ↗

HIỂU CƠ CHẾ

contributing.md hướng dẫn mở trang danh sách, chỉnh README qua giao diện GitHub, mô tả lý do và gửi PR. Tuy vậy, người học cần đọc hướng dẫn của chính danh sách đích: các repo khác nhau có thể có quy tắc khác. Đề xuất nên nhỏ, chỉ rõ mục thêm hoặc sửa và bằng chứng vì sao phù hợp. Nếu maintainer yêu cầu điều chỉnh, sửa cùng PR theo phản hồi. Bài học này dạy cách chuẩn bị; giáo trình không tự gửi PR tới awesome hay các danh sách bên ngoài. Khi repo tạm dừng nhận PR, giữ bản nháp của bạn và theo dõi thông báo. [R3]

LÀM THEO TỪNG BƯỚC

1. Mở contributing.md của danh sách đích và kiểm tra thông báo tiếp nhận hiện tại, issue và PR để tránh trùng.
2. Chuẩn bị một entry hoặc sửa một lỗi có bằng chứng. Ghi vị trí và lý do, chưa mở PR khi vẫn đang hoàn thiện nội dung.
3. Dùng trình duyệt chỉnh README trên nhánh/fork của bạn, xem diff và Preview để bảo đảm chỉ thay đổi điều dự định.
4. Khi repo cho phép và bạn đã tự kiểm tra, gửi PR có mô tả rõ. Đọc phản hồi và cập nhật cùng đề xuất khi cần.

ĐẦU RA CẦN ĐẠT

Một bản diff nhỏ và mô tả PR có lý do; gửi thực tế chỉ khi đủ điều kiện.

LỖI THƯỜNG GẶP

Thêm một công cụ vào repo tổng thay vì danh sách chuyên ngành, hoặc mở nhiều PR trùng nội dung.

TỰ KIỂM TRA

PR bị yêu cầu sửa có nghĩa đóng góp thất bại không?

Gợi ý: Không. Phản hồi là một phần của việc hoàn thiện; cần xác minh và sửa hoặc giải thích bằng bằng chứng.

Làm thêm: Viết mô tả PR gồm điều thay đổi, lý do phù hợp và cách bạn đã kiểm chứng.

Kiểm tra điều kiện cho một danh sách mới

MỤC TIÊU / Biết khi nào chưa nên gửi danh sách vào repo tổng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R5] ↗

HIỂU CƠ CHẾ

create-list.md yêu cầu chờ ít nhất 30 ngày. Checklist làm rõ cách tính: từ commit thực sự đầu tiên hoặc khi công khai, lấy mốc gần hơn. Checklist cũng yêu cầu danh sách không do AI tạo, PR không hoàn toàn do AI tạo, lint đạt và review ít nhất bốn PR khác bằng phản hồi có nội dung; bình luận "looks good" không được tính. Đây là quy tắc tại thời điểm nguồn được đọc, có thể thay đổi. Bộ khung và ví dụ AI hỗ trợ trong giáo trình không phải danh sách đủ điều kiện để bạn gửi nguyên trạng. Cần công sức tuyển chọn và kiểm chứng thật của người duy trì. [R4][R5]

LÀM THEO TỪNG BƯỚC

1. Mở lại checklist gốc và kiểm tra trạng thái tiếp nhận trước mọi hành động gửi.
2. Xác minh phạm vi không trùng, tên repo/nhánh/tiêu đề/mục lục/giấy phép/định dạng và kết quả lint theo từng yêu cầu.
3. Đối chiếu ngày bắt đầu thực sự và ngày công khai; không dùng một commit khung để giả lập thời gian trưởng thành.
4. Tự đánh giá công sức tuyển chọn và hoàn tất nghĩa vụ review theo hướng dẫn. Chỉ chuẩn bị đề xuất khi mọi điều kiện thật sự đạt.

ĐẦU RA CẦN ĐẠT

Một bảng điều kiện có bằng chứng; các mục chưa đạt phải giữ trạng thái chưa sẵn sàng.

LỖI THƯỜNG GẶP

Coi 30 ngày là điều kiện duy nhất, hoặc tự động tạo hàng loạt PR để đủ số review.

TỰ KIỂM TRA

Vì sao giáo trình không tạo một danh sách AI hoàn chỉnh rồi gửi giúp bạn?

Gợi ý: Vì chính sách nguồn yêu cầu không phải danh sách do AI tạo và cần đánh giá, đầu tư thật từ người duy trì.

Làm thêm: Tìm ba điều kiện mà bộ bài tập hiện tại chưa thể chứng minh. Viết kế hoạch đáp ứng chúng bằng công việc của chính bạn.

moda.app

Duy trì và tự đánh giá

Làm sao danh sách vẫn có ích sau ngày tạo?

Tài nguyên thay đổi, nhu cầu người đọc thay đổi và các liên kết có thể mất tác dụng. Một danh sách tốt cần được duy trì bằng quyết định có lý do, chứ không chỉ nhận thêm liên kết. Manifesto nhấn mạnh tôn trọng ý kiến người khác; hướng dẫn chất lượng giới hạn những tài nguyên lỗi thời trong danh sách chính. Chương cuối đề xuất nhịp kiểm tra, nhật ký quyết định và một đồ án tổng hợp. Nhịp và thang tự chấm là đề xuất học tập, không phải quy định chính thức của Awesome. [R2][R5]

Tìm
Xác định nhu cầu

→

Kiểm chứng
Ghi bằng chứng

→

Chọn
Giải thích quyết định

BA KHÁI NIỆM CẦN GIỮ LẠI

Maintenance	Kiểm tra và cập nhật nội dung theo thời gian.
Decision log	Nhật ký lý do thêm, sửa, giữ hoặc loại.
Capstone	Bài thực hành tổng hợp cuối lộ trình.

Thiết kế vòng bảo trì

MỤC TIÊU / Có cách phát hiện và xử lý tài nguyên không còn phù hợp.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R2] ↗

HIỂU CƠ CHẾ

Hãy xem danh sách như một sản phẩm có người dùng. Sau mỗi phản hồi hoặc lần kiểm tra, bạn có thể thêm, sửa, giữ hoặc loại một mục; quyết định nên có ngày và bằng chứng. Người viết đề xuất ưu tiên entry được dùng nhiều, có báo lỗi hoặc có điều kiện sử dụng dễ thay đổi. Một liên kết bị lỗi một lần có thể là sự cố tạm thời, nên cần phân biệt lỗi truy cập với tài nguyên đã ngừng hoạt động. Nếu cần giữ mục lịch sử, hướng dẫn Awesome cho phép đặt các mục không phù hợp danh sách chính vào tệp riêng. [R2][R5]

LÀM THEO TỪNG BƯỚC

1. Chọn lịch kiểm tra phù hợp với công sức bạn thực sự có; ghi đó là đề xuất của bạn, không phải yêu cầu Awesome.
2. Ưu tiên mục có phản hồi hoặc bằng chứng cũ. Mở lại URL, tài liệu và tình trạng dự án.
3. Ghi quyết định giữ/sửa/hoãn/loại, nêu bằng chứng và ảnh hưởng tới người đọc; kiểm tra lại khi nghi sự cố tạm thời.
4. Khi người khác phản đối, đối chiếu phạm vi và bằng chứng thay vì tranh luận bằng mức phổ biến. Cập nhật quy tắc nếu cần.

ĐẦU RA CẦN ĐẠT

Một bảng bảo trì và nhật ký cho ít nhất một quyết định có thể kiểm tra lại.

LỖI THƯỜNG GẶP

Chỉ thêm tài nguyên mới mà không xem mục cũ, hoặc xóa ngay khi có một lỗi mạng đơn lẻ.

TỰ KIỂM TRA

Vì sao nên giữ nhật ký của một mục đã loại?

Gợi ý: Để giải thích quyết định, tránh thêm lại cùng vấn đề và biết khi nào có lý do xem xét lại.

Làm thêm: Giả lập một entry thiếu tài liệu sau thay đổi. Viết quyết định tạm thời và điều kiện để đưa nó trở lại.

Đồ án: danh sách nhỏ có bằng chứng

MỤC TIÊU / Hoàn thành một bộ tài liệu người khác có thể dùng và kiểm chứng.

Bài đọc trên GitHub Radar ↗ · Tài liệu gốc [R5] ↗

HIỂU CƠ CHẾ

Đồ án tổng hợp không chấm bằng số entry. Mục tiêu là chứng minh bạn xác định đúng nhu cầu, tìm được tài nguyên, tự thử, viết rõ và biết giới hạn. Bộ nộp đề xuất gồm README, hồ sơ ứng viên, quy tắc đóng góp, quyết định giấy phép và nhật ký kiểm tra. Nếu chưa đủ điều kiện upstream, hãy giữ nó là dự án học tập cá nhân và ghi rõ trạng thái. Một danh sách ít mục nhưng có bằng chứng sẽ phản ánh kỹ năng tốt hơn một bản tổng hợp lớn chưa ai kiểm chứng. Kết quả này phục vụ người học; không tự động chứng minh bạn đã thành thạo mọi lĩnh vực trong 27 nhóm. [R2][R5]

LÀM THEO TỪNG BƯỚC

1. Chọn phạm vi nhỏ và hoàn thành các tệp trong bộ bài tập bằng nội dung bạn tự đọc, thử và đánh giá.
2. Nhờ một người dùng thực hiện một nhiệm vụ tìm kiếm. Lưu điều họ hiểu sai và bản sửa sau phản hồi.
3. Chạy checklist nội dung và lint nếu môi trường cho phép. Giữ trung thực các mục chưa xác minh hoặc chưa chạy.
4. Tự chấm theo rubric cuối tài liệu, rồi viết điều đã làm được, giới hạn còn lại và lần kiểm tra tiếp theo.

ĐẦU RA CẦN ĐẠT

Một danh sách cá nhân có thể đọc, một hồ sơ bằng chứng và bản tự đánh giá.

LỖI THƯỜNG GẶP

Tuyên bố "đạt chuẩn Awesome" chỉ vì đủ tệp mẫu. Tính đầy đủ cấu trúc khác với chất lượng và điều kiện được tiếp nhận.

TỰ KIỂM TRA

Dấu hiệu bạn đã học được cách dùng Awesome là gì?

Gợi ý: Bạn tìm được đúng lối vào, tự đánh giá tài nguyên và giải thích được vì sao chọn hoặc loại một mục.

Làm thêm: Trình bày dự án cho một người khác mà không mở giáo trình. Sau đó dùng checklist để tìm phần bạn còn giải thích chưa rõ.

moda.app

Thuật ngữ & rubric tự đánh giá

README / mục giới thiệu	Nơi hiểu phạm vi và tìm đường đi của repo.
Entry / mục tài nguyên	Tên + liên kết + mô tả chức năng.
Curation / tuyển chọn	Chọn có lý do, không chỉ thu thập thật nhiều.
Evidence / bằng chứng	Nguồn và kết quả có thể mở lại để kiểm chứng.
PR / đề xuất thay đổi	Cách mời maintainer xem xét nội dung của bạn.
Lint / kiểm định dạng	Công cụ hỗ trợ; không thay đánh giá nội dung.

RUBRIC CỦA NGƯỜI VIẾT / 0–2 ĐIỂM MỖI MỤC

1. Phạm vi rõ và tránh trùng.
2. Bằng chứng cho từng entry.
3. Mô tả giúp lựa chọn.
4. Mục lục và quy tắc dễ dùng.
5. Kiểm tra và giới hạn trung thực.
6. Có phản hồi và kế hoạch duy trì.

0: chưa có · 1: có một phần · 2: có bằng chứng.
Không có điểm số nào tự chứng nhận được nhận vào Awesome.

moda.app

Bộ bài tập đi kèm

Tệp mẫu trong thư mục lab/ là khung để bạn tự điền. Không gửi nguyên trạng thành một danh sách được tuyên bố đủ chuẩn.

project-brief.md	Mục tiêu, người đọc, ràng buộc, đầu ra và ngoài phạm vi.
candidate-log.csv	URL, tác vụ đã thử, ngày, bằng chứng và hạn chế.
readme.template.md	Bố cục danh sách; còn cần nội dung tuyển chọn thật.
contributing.template.md	Tiêu chí nhận/loại, bằng chứng và quy trình đề xuất.
decision-log.md	Ngày, mục thay đổi, lý do và điều kiện xem lại.

LỆNH LINT TỪ TÀI LIỆU GỐC / CẦN NODE.JS VÀ GIT

```
npx awesome-lint
```

Chạy trong thư mục repo của chính bạn; lệnh có thể tải gói để thực thi. Nếu chưa có môi trường, ghi rõ “chưa chạy”, rồi dùng checklist thủ công. [R6]

Trước khi đề xuất upstream

Đây là tóm tắt để tự kiểm tra, không thay checklist gốc. Đọc lại trạng thái tiếp nhận trước khi gửi. Bài tập AI hỗ trợ không phải danh sách đủ điều kiện gửi nguyên trạng.

- [ ] **Phạm vi & công sức**

  Không trùng danh sách tốt đã có; bạn thật sự đọc, thử và tuyển chọn. Danh sách không do AI tạo; PR không hoàn toàn do AI tạo.

- [ ] **Thời gian trưởng thành**

  Ít nhất 30 ngày từ commit thực sự đầu tiên hoặc lúc công khai, lấy mốc gần hơn.

- [ ] **Cấu trúc & chất lượng**

  Tên repo dạng slug, nhánh main, Contents, mô tả, badge, hướng dẫn đóng góp và giấy phép phù hợp.

- [ ] **Kiểm tra & cộng đồng**

  Chạy awesome-lint; thực hiện review có nội dung cho ít nhất 4 PR khác theo hướng dẫn, không chỉ “looks good”.

- [ ] **Định dạng đề xuất**

  Tiêu đề Add Name of List; entry mô tả chủ đề, không quảng cáo. Thêm ở cuối nhóm phù hợp; URL có #readme.

- [ ] **Chính sách hiện hành**

  Checklist hiện không nhận danh sách blockchain mới dù README còn các mục lịch sử. Kiểm tra mọi yêu cầu gốc, gồm chi tiết xác nhận đã đọc.

Nguồn gốc & phạm vi kiểm chứng

R1 / README: bản đồ nhóm và liên kết

https://github.com/sindresorhus/awesome/blob/bc98e517ddca672f55f9857d714fc3ea3c3540b2/readme.md

R2 / The awesome manifesto

https://github.com/sindresorhus/awesome/blob/bc98e517ddca672f55f9857d714fc3ea3c3540b2/awesome.md

R3 / Contribution Guidelines

https://github.com/sindresorhus/awesome/blob/bc98e517ddca672f55f9857d714fc3ea3c3540b2/contributing.md

R4 / Creating Your Own List

https://github.com/sindresorhus/awesome/blob/bc98e517ddca672f55f9857d714fc3ea3c3540b2/create-list.md

R5 / Checklist đề xuất danh sách

https://github.com/sindresorhus/awesome/blob/bc98e517ddca672f55f9857d714fc3ea3c3540b2/pull_request_template.md

R6 / awesome-lint: CLI và kiểm tra

https://github.com/sindresorhus/awesome-lint/blob/4f870abda373cfa18c67822c39ee29c451ae8a70/readme.md

Nguồn được đọc ngày 24/10/2026 và ghim theo commit. Giáo trình không chạy thử toàn bộ tài nguyên trong các danh sách. Các chương, bài tập và rubric là nội dung biên soạn riêng.